

UNIT – 1
ASSESSMENT – 1
REPORT

1. Title:

Water Jug Problem using State Space Search

Objective:

To solve the Water Jug Problem using Artificial Intelligence state space search techniques.

Theory:

The Water Jug Problem is a classical Artificial Intelligence search problem.

The objective is to reach a desired goal state from an initial state by applying a sequence of legal operations.

Each possible amount of water inside the two jugs represents a unique state.

The solution is obtained by searching through the state space until the goal state is reached.

Procedure:

1. Start with both jugs empty.
2. Fill the 3-gallon jug.
3. Transfer water into the 4-gallon jug.
4. Fill the 3-gallon jug again.
5. Pour into the 4-gallon jug until it becomes full.
6. Empty the 4-gallon jug.
7. Transfer the remaining 2 gallons into the 4-gallon jug.
8. Stop when the goal state is reached.

Input:

```
from collections import deque
```

```
def water_jug_bfs(jug1_cap, jug2_cap, target):
    start = (0, 0)
    visited = set([start])
    queue = deque([(start, [start])])

    while queue:
        (x, y), path = queue.popleft()

        if x == target or y == target:
            return path

        next_states = [
            (jug1_cap, y),      # Fill jug1
            (x, jug2_cap),      # Fill jug2
            (0, y),             # Empty jug1
            (x, 0),             # Empty jug2
            (x - min(x, jug2_cap - y), y + min(x, jug2_cap - y)), # Pour
jug1 -> jug2
            (x + min(y, jug1_cap - x), y - min(y, jug1_cap - x)) # Pour
jug2 -> jug1
        ]

        for state in next_states:
            if state not in visited:
                visited.add(state)
                queue.append((state, path + [state]))

    return None

# Solve: 4-gallon and 3-gallon jugs, target = 2 gallons
solution = water_jug_bfs(4, 3, 2)
print("Water Jug Solution Path (jug1, jug2):")
for step, state in enumerate(solution):
    print(f"Step {step}: {state}")
```

Output:

```
==== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/qn1.py ====  
Water Jug Solution Path (jug1, jug2):  
Step 0: (0, 0)  
Step 1: (0, 3)  
Step 2: (3, 0)  
Step 3: (3, 3)  
Step 4: (4, 2)  
>>>
```

Result:

The Water Jug Problem was successfully solved.

The required goal state **(2,0)** was reached, proving that exactly **2 gallons of water** can be obtained in the 4-gallon jug using the permitted operations.

2. Title:

Study of Mars Rover as an Intelligent Agent

Objective:

To understand the working of the Mars Rover by analyzing its percepts, operating environment, actions, performance measures, and suitable AI agent architecture.

Theory:

A Mars Rover is an autonomous robotic system designed to explore the Martian surface. It uses sensors, cameras, scientific instruments, and Artificial Intelligence algorithms to navigate unknown environments, collect scientific data, and communicate with Earth.

Since communication delays between Earth and Mars are significant, the rover must make many decisions independently.

Artificial Intelligence enables the rover to perceive its surroundings, avoid obstacles, plan routes, and adapt to changing environmental conditions.

Applications:

- Space exploration
- Planetary research
- Autonomous robots
- Disaster response
- Deep-sea exploration
- Military reconnaissance

Input:

```
import random
```

```
class MarsRoverAgent:  
    def __init__(self):
```

```

self.position = (0, 0)
self.battery = 100
self.samples_collected = 0
self.internal_map = {} # model of the world (model-based agent)

def perceive(self):
    percept = {
        "terrain": random.choice(["rocky", "sandy", "flat"]),
        "obstacle_ahead": random.choice([True, False]),
        "sample_detected": random.choice([True, False]),
        "battery_level": self.battery
    }
    return percept

def update_model(self, percept):
    self.internal_map[self.position] = percept["terrain"]

def decide_action(self, percept):
    if percept["battery_level"] < 20:
        return "return_to_charge"
    if percept["obstacle_ahead"]:
        return "navigate_around"
    if percept["sample_detected"]:
        return "collect_sample"
    return "move_forward"

def act(self, action):
    if action == "collect_sample":
        self.samples_collected += 1
        self.battery -= 5
    elif action == "move_forward":
        self.position = (self.position[0] + 1, self.position[1])
        self.battery -= 3
    elif action == "navigate_around":
        self.battery -= 4
    elif action == "return_to_charge":
        self.battery = 100
    print(f"Action: {action} | Position: {self.position} | Battery: {self.battery} | Samples: {self.samples_collected}")

def run(self, steps=10):

```

```
    for _ in range(steps):
        percept = self.perceive()
        self.update_model(percept)
        action = self.decide_action(percept)
        self.act(action)

rover = MarsRoverAgent()
rover.run(10)
```

Output:

```
>>> ===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/qn1.py =====
Action: navigate_around | Position: (0, 0) | Battery: 96 | Samples: 0
Action: navigate_around | Position: (0, 0) | Battery: 92 | Samples: 0
Action: collect_sample | Position: (0, 0) | Battery: 87 | Samples: 1
Action: collect_sample | Position: (0, 0) | Battery: 82 | Samples: 2
Action: collect_sample | Position: (0, 0) | Battery: 77 | Samples: 3
Action: collect_sample | Position: (0, 0) | Battery: 72 | Samples: 4
Action: move_forward | Position: (1, 0) | Battery: 69 | Samples: 4
Action: move_forward | Position: (2, 0) | Battery: 66 | Samples: 4
Action: navigate_around | Position: (2, 0) | Battery: 62 | Samples: 4
Action: move_forward | Position: (3, 0) | Battery: 59 | Samples: 4
>>> |
```

Result:

The Mars Rover was successfully analyzed as an intelligent AI agent. Its percepts, operating environment, actions, performance measures, and learning agent architecture were studied, demonstrating how Artificial Intelligence enables autonomous exploration of Mars.

3. Title:

8-Queens Problem using Backtracking Search Algorithm

Objective:

To solve the 8-Queens Problem using the Backtracking Search Algorithm and understand search space exploration in Artificial Intelligence.

Theory:

The 8-Queens Problem is a classical AI search problem.

The objective is to place eight queens on a chessboard without allowing any queen to attack another queen.

The solution is obtained using the **Backtracking Search Algorithm**, which explores the search space systematically.

Whenever a queen cannot be placed safely, the algorithm removes the previously placed queen and tries another position.

This process continues until a valid arrangement is found.

Input:

```
def is_safe(board, row, col, n):  
    for i in range(row):  
        if board[i] == col or \  
            abs(board[i] - col) == abs(i - row):  
            return False  
    return True  
  
def solve_n_queens(board, row, n, solutions):  
    if row == n:  
        solutions.append(board[:])
```

```

        return
    for col in range(n):
        if is_safe(board, row, col, n):
            board[row] = col
            solve_n_queens(board, row + 1, n, solutions)
            board[row] = -1 # backtrack

def print_board(solution, n):
    for row in range(n):
        line = ""
        for col in range(n):
            line += "Q " if solution[row] == col else ". "
        print(line)
    print()

n = 8
board = [-1] * n
solutions = []
solve_n_queens(board, 0, n, solutions)

print(f"Total solutions found: {len(solutions)}")
print("\nFirst solution:")
print_board(solutions[0], n)

```


Output:

```
>>> ===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/qnl.py =====
Total solutions found: 92

First solution:
Q . . . . .
. . . . Q . .
. . . . . Q .
. . . . Q . .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .
```

Result:

The 8-Queens problem was successfully solved using a Backtracking Search algorithm. The algorithm places queens row by row and backtracks whenever a conflict occurs. The program found 92 valid solutions, with the first solution being [0, 4, 7, 5, 2, 6, 1, 3]. This confirms that backtracking effectively finds valid non-attacking queen placements.

4. Title:

Problem Solving Agent for OLA Cab Booking Application

Objective:

To understand how a Goal-Based Agent is used in an online cab booking application and to develop the pseudocode for the booking process.

Theory:

Artificial Intelligence agents are systems that perceive their environment and perform actions to achieve a specific goal.

In the OLA Cab Booking application, the user's goal is to travel from one location to another safely and efficiently.

The application collects information such as pickup location, destination, available cab types, estimated fare, and waiting time. Based on this information, the system helps the customer choose a suitable cab and completes the booking.

Since the objective is to achieve a clearly defined goal, the application behaves like a **Goal-Based Agent**.

Input:

```
class CabBookingAgent:
```

```
    def __init__(self):
```

```
        self.cab_options = {
```

```
            "micro": {"base_fare": 40, "per_km": 8, "wait_time": 5},
```

```
            "mini": {"base_fare": 50, "per_km": 10, "wait_time": 4},
```

```
            "sedan": {"base_fare": 80, "per_km": 14, "wait_time": 6},
```

```
            "shared": {"base_fare": 25, "per_km": 5, "wait_time": 10},
```

```
            "prime": {"base_fare": 100, "per_km": 18, "wait_time": 3},
```

```
}
```

```
def calculate_fare(self, cab_type, distance_km):  
    cab = self.cab_options[cab_type]  
    return cab["base_fare"] + cab["per_km"] * distance_km  
  
def evaluate(self, cab_type, distance_km, priority="cost"):  
    fare = self.calculate_fare(cab_type, distance_km)  
    wait = self.cab_options[cab_type]["wait_time"]  
  
    if priority == "cost":  
        return -fare # lower fare = better score  
    elif priority == "time":  
        return -wait # lower wait = better score  
    else:  
        return -(fare * 0.6 + wait * 0.4) # balanced  
  
def book_best_cab(self, distance_km, priority="cost"):  
    best_cab, best_score = None, float("-inf")  
    for cab_type in self.cab_options:  
        score = self.evaluate(cab_type, distance_km, priority)  
        if score > best_score:  
            best_score = score  
            best_cab = cab_type  
  
    fare = self.calculate_fare(best_cab, distance_km)
```

```
    print(f'Recommended cab: {best_cab.upper()} | Estimated Fare:  
Rs. {fare} | ETA: {self.cab_options[best_cab]['wait_time']} min")  
  
    return best_cab
```

Example run

```
agent = CabBookingAgent()  
agent.book_best_cab(distance_km=12, priority="cost")  
agent.book_best_cab(distance_km=12, priority="time")
```

Output:

```
>>>|==== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/qnl.py ====  
Recommended cab: SHARED | Estimated Fare: Rs.85 | ETA: 10 min  
Recommended cab: PRIME | Estimated Fare: Rs.316 | ETA: 3 min
```

Advantages:

- Helps users reach their destination efficiently.
- Provides multiple cab options.
- Reduces travel time.
- Easy and user-friendly booking process.
- Optimizes route selection and cab allocation.

Applications:

- OLA
- Uber
- Rapido
- Taxi booking services
- Ride-sharing applications

Result:

The OLA Cab Booking problem was successfully analyzed. A **Goal-Based Agent** was identified as the most suitable problem-solving agent because it focuses on achieving the customer's goal of reaching the destination. The pseudocode and Python implementation demonstrate the booking process effectively.

5. Title:

Uniform Cost Search (UCS) for Finding the Least-Cost Delivery Path

Objective:

To determine the least-cost path between the starting warehouse **S** and the destination warehouse **G** using the **Uniform Cost Search (UCS)** algorithm.

Theory:

Uniform Cost Search (UCS) is an uninformed search algorithm used to find the path with the minimum cumulative cost.

Instead of expanding nodes level by level like Breadth-First Search, UCS always selects the node with the **lowest total path cost** from the priority queue.

It guarantees the optimal solution when all edge costs are positive.

Algorithm:

1. Insert the start node into the priority queue with cost **0**.
2. Remove the node having the smallest cumulative cost.
3. If it is the goal node, stop.
4. Otherwise, expand its neighbors.
5. Add neighbors to the priority queue with updated cumulative costs.
6. Repeat until the goal is found.

Input:

```
import heapq
```

```
def ucs(graph, start, goal):
```

```
    frontier = [(0, start, [start])] # (cost, node, path)
```

```
visited = {}
```

```
while frontier:
```

```
    cost, node, path = heapq.heappop(frontier)
```

```
    if node == goal:
```

```
        return path, cost
```

```
    if node in visited and visited[node] <= cost:
```

```
        continue
```

```
    visited[node] = cost
```

```
    for neighbor, weight in graph.get(node, []):
```

```
        new_cost = cost + weight
```

```
        if neighbor not in visited or new_cost <
visited.get(neighbor, float('inf')):
```

```
            heapq.heappush(frontier, (new_cost, neighbor, path +
[neighbor]))
```

```
    return None, float('inf')
```

```
# Graph from the figure
```

```
graph = {
```

```
    "S": [("A", 1), ("G", 12)],
```

```
    "A": [("B", 3), ("C", 1)],
```

```
    "B": [("D", 3)],
```

```
    "C": [("D", 1), ("G", 2)],
```

```
    "D": [("G", 3)],
```

```

    "G": []
}

path, cost = ucs(graph, "S", "G")
print(f"Least-cost path: {' -> '.join(path)}")
print(f"Minimum cost: {cost}")

```

Output:

```

>>> ===== RESTART: C:/Users/DELL/AppData/Local/Programs/Python/Python313/qn1.py =====
Least-cost path: S -> A -> C -> G
Minimum cost: 4
>>>

```

Advantages:

- Finds the optimal (least-cost) path.
- Suitable for weighted graphs.
- Guarantees correctness for positive edge costs.
- Widely used in AI search problems.

Applications:

- GPS Navigation
- Google Maps
- Delivery Route Planning
- Robotics
- Network Routing
- Logistics and Supply Chain Management

Result:

The Uniform Cost Search algorithm successfully determined the least-cost path from **S** to **G**.